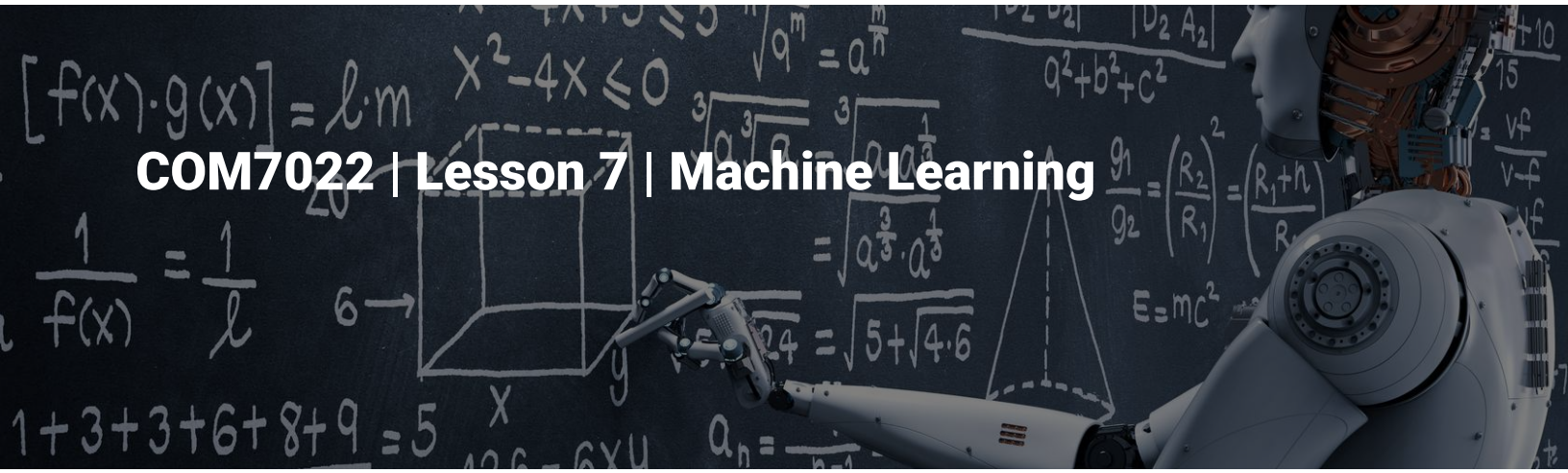


COM7022 | Lesson 7 | Machine Learning

Machine Learning (COM7022)

Lesson 7: Support vector machine (SVM)

Cover image by Phonlamai Photo from Shutterstock.com

- Lesson introduction
- Topic 1: What is SVM?
- Topic 2: Building an SVM Model
- Topic 3: Support Vector Regression
- Topic 4: Summarising SVM and SVR
- Knowledge check
- Lesson summary

Reading List

Lesson introduction

Welcome to Lesson 7

LESSON INTRODUCTION | LEARNING OUTCOMES | ACCESSIBILITY GUIDANCE

Welcome to Lesson 7

Within this lesson we shall be taking a deep dive into Support Vector Machine (SVM) and Support Vector Regression (SVR): both methodologies very different, but similar in approach. Although this may seem confusing to begin with, as we progress through the various topic, not doubt things will become much clearer.

Both SVM and SVR are complex topics, yet they are widely used within a machine learning environment for initial classification purposes. Due to the complexity of the topics, full coded examples are given for each ideology.

We have a lot to get through within this first topic, so let us dive straight in.

See you in the first topic!

Computing

At Arden University

LESSON INTRODUCTION | LEARNING OUTCOMES | ACCESSIBILITY GUIDANCE

Upon completing this lesson you will:

- Know what SVM is and what it is used for.
- Be able to create a SVM investigation yourself.
- Fully understand what SVR is and what it is used for.
- Confidently be able to self-summarise the two applications and any differences.

Lesson 7 aligns to Learning Objectives 2 and 3:

- **Select and apply appropriate machine learning techniques for a given scenario.**
- **Critically evaluate the results from the application of Machine Learning techniques and propose improvements.**

Computing
At Arden University

LESSON INTRODUCTION | LEARNING OUTCOMES | ACCESSIBILITY GUIDANCE

We have worked across teams at Arden University to try and create an accessible and inclusive experience within the learning content. We are continuously assessing the accessibility of our lessons to make improvements and enhancements based on testing and feedback.

This lesson may include screenshots of code and software used. Please be aware that these may not always have a text alternative, and that other features such as size and colour may not be accessible.

If you experience any issues with accessing the learning content throughout this module, there are several support services in place to help you.

- Arden University has a dedicated Inclusion Services Team who can help you with getting adjustments and equipment.
- The Library Portal also has information on the accessibility tools that can help you access resources: Accessibility Tools – Library Portal
- If you are struggling with accessing a specific part of the content e.g. a video resource that is missing captions, please speak to your Lecturer who may be able to advise on an alternative resource in the interim to ensure you are able to get the learning you need to progress with the lesson.

Continue to Topic 1

Topic 1: What is SVM?

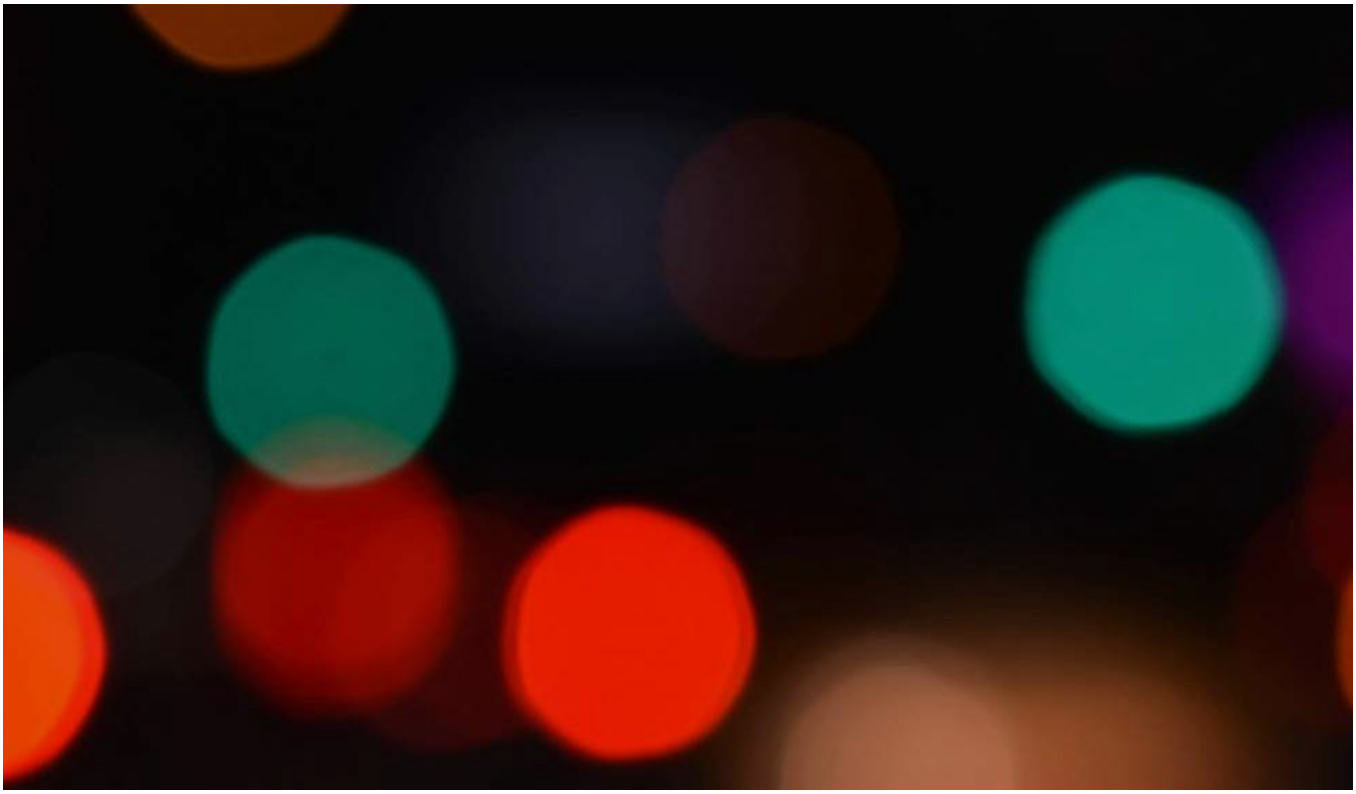


Image from Articulate Rise 360 Content Library.

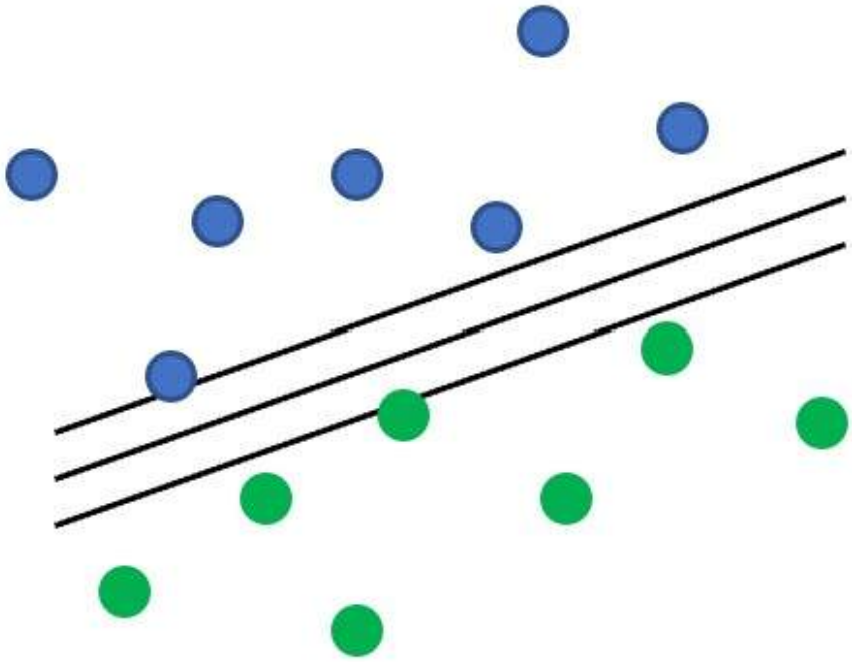


Figure 7.01. SVM holistic graphical representation by Arden University.

First and foremost, a Support Vector Machine (SVM) investigation is primarily a classification study. When detailing SVM, there are certain incremental improvements that can be made upon the original classification model that will eventually bring us to SVM within a machine learning environment. For us to fully understand SVM in its entirety, it is well worth our time to go back to an elementary classification problem in which to build upon. Therefore, say for example we have a dataset which recorded dogs' weight and we wanted to know that once the model was built, if we add a new data point, would the model categories correctly, i.e., ideal weight or overweight. Therefore, let us begin with a fictitious dataset, as shown below in Table 7.01 and then represented graphically in Figure 7.02.

ID	Breed	Weight (KG)	Ideal / Over
1	Poodle	23	23
2	G. Shepard	46	46
3	Labrador	38	38
4	F. Bulldog	20	20
5	E. Bulldog	26	26
6	Spaniel	18	18
7	Dobermann	25	25
8	B. Terrier	22	22
9	J. Russell	18	18
10	Poodle	29	29
11	G. Shepard	50	50
12	Labrador	37	37
13	F. Bulldog	32	32
14	E. Bulldog	37	37
15	Spaniel	22	22
16	Dobermann	44	44

Table 7.01. (Accessible table).

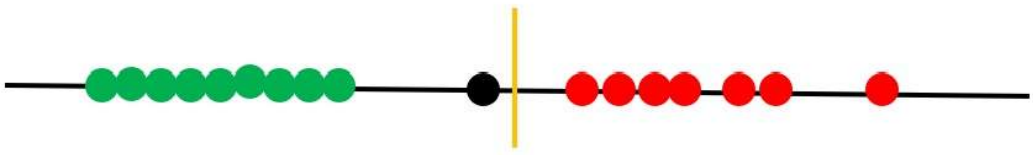


Figure 7.02. Data representation of Table 7.01 of ideal (green) and overweight (red) dogs by Arden University.

It would be intuitive to place the threshold for categorisation in the middle of the 1-D line, which is what we have included above in orange. Now, when we get a new data entry, the initial classification will be able to categorise the correct weight, whether that be ideal, or over. Right?

Well, what about if a new data entry was introduced and was categorised as being ideal, as Figure 7.03 shows. Could we be correct that the model has categorised the new entry correctly though, as it looks like the data entry is much closer to the over category, than to the ideal? This is where the model would fail using the middle threshold line, as quite clearly, the new data entry is much closer to the over category. So, the first accuracy improvement would be, how to we improve the measure of the threshold line? What about if we took the average distance from the two closest over and ideal data points and moved the threshold line there? Let's try it and see if this improves the accuracy of the model. This can be seen in Figure 7.03.

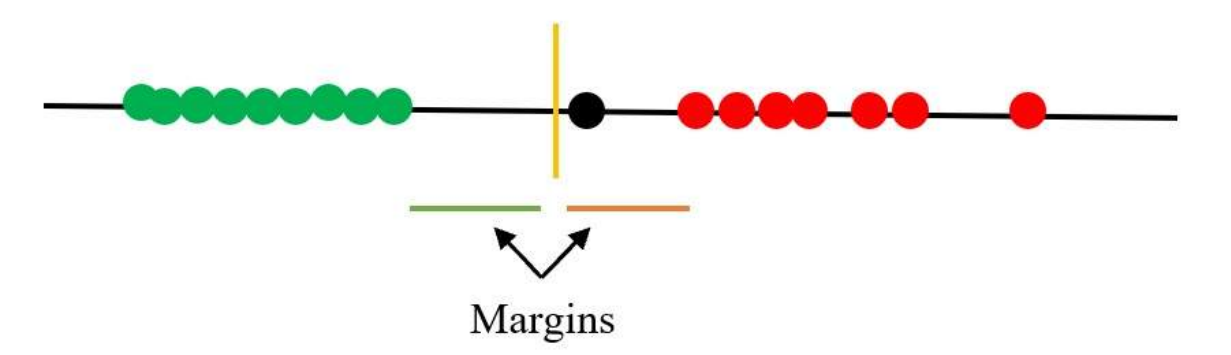


Figure 7.03. Improving the accuracy of the model using margins by Arden University.

As can be clearly seen, by adopting the average distance from the closest two data points from each category, it has improved the accuracy of the model. In using this approach, we have identified the shortest distance between the data points and the threshold, therefore we have created margins, which are defined also in Figure 7.04. The margin used is correctly defined as a Maximal Margin Classifier (MMC).

However, as with most methodologies within machine learning, the MMC is not without its problems. Say for example we had a dataset that contained an outlier, as most datasets do, the

classification of that outlier would possibly be categorised incorrectly, as Figure 7.04 demonstrates. How do we improve upon the MMC then, so that we the model can be yet again more accurate?

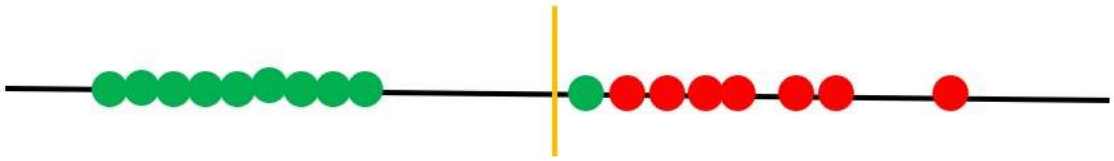


Figure 7.04. Incorrect categorisation of a data point by Arden University.

Before we answer the above question, if we did nothing, the observations would be very close to the MMC. Plus, if a new data point was introduced, that too would be miss categorised. This would mean that MMCs are very sensitive to outliers within the training data, as shown below in Figure 7.05.

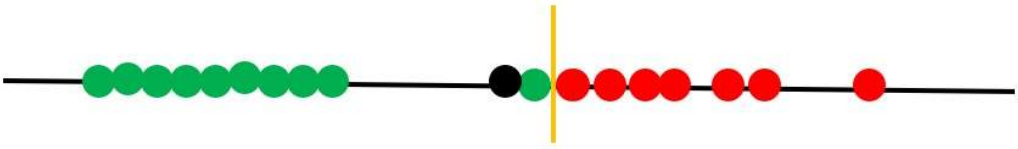


Figure 7.05. Demonstrating the sensitivity of the MMC by Arden University.

Referring back to our previous question, we can improve the MMC by incorporating misclassification: a little bit counter intuitive, but hopefully as we explain things will make a little more sense. This is a classic example of the bias/variance trade off that is experienced by most, if not all machine learning models. As an example of this trade off, before we used misclassification, the threshold was very sensitive to outliers, therefore inherently included low bias. Conversely, when the new data point was added, it misclassified, exhibiting high variance, as shown in Figure 7.05.

By including misclassification, it allows for a much greater accuracy in the long run, the caveat, as you will have undoubtedly determined, is that we will not have a model that is 100% accurate, or a module that has an accuracy between 96% and 98%. However, in the long run, the model will be far more accurate and far less sensitive to outliers if misclassification had not been included. When a machine learning analyst takes into account misclassification, the distance between the threshold and the data points are called the “soft margins”. However, this also exposes yet another question, “How do we determine what the correct soft margins are?”. The answer to this is cross validation. In other words, we iterate the model, altering the margins until we have reached the maximum accuracy that can be attained. When the optimum threshold has been determined through the iterative process of soft margins, then the result is known as the Support Vector Classifier (SVC).

Up until this point, we have only been dealing with data that is one dimensional, in other words, the data has only been concerned with dogs’ weight. However, data that is one dimensional is rare and not very intuitive, so let us add in another dimension to the data, say the dogs’ height. When we introduce a 2nd dimension, we could end up with a 2-D graph similar to Figure 7.06 below, where the SVC is represented as a line (black) and the soft margins are represented in blue, between the two closest data points.

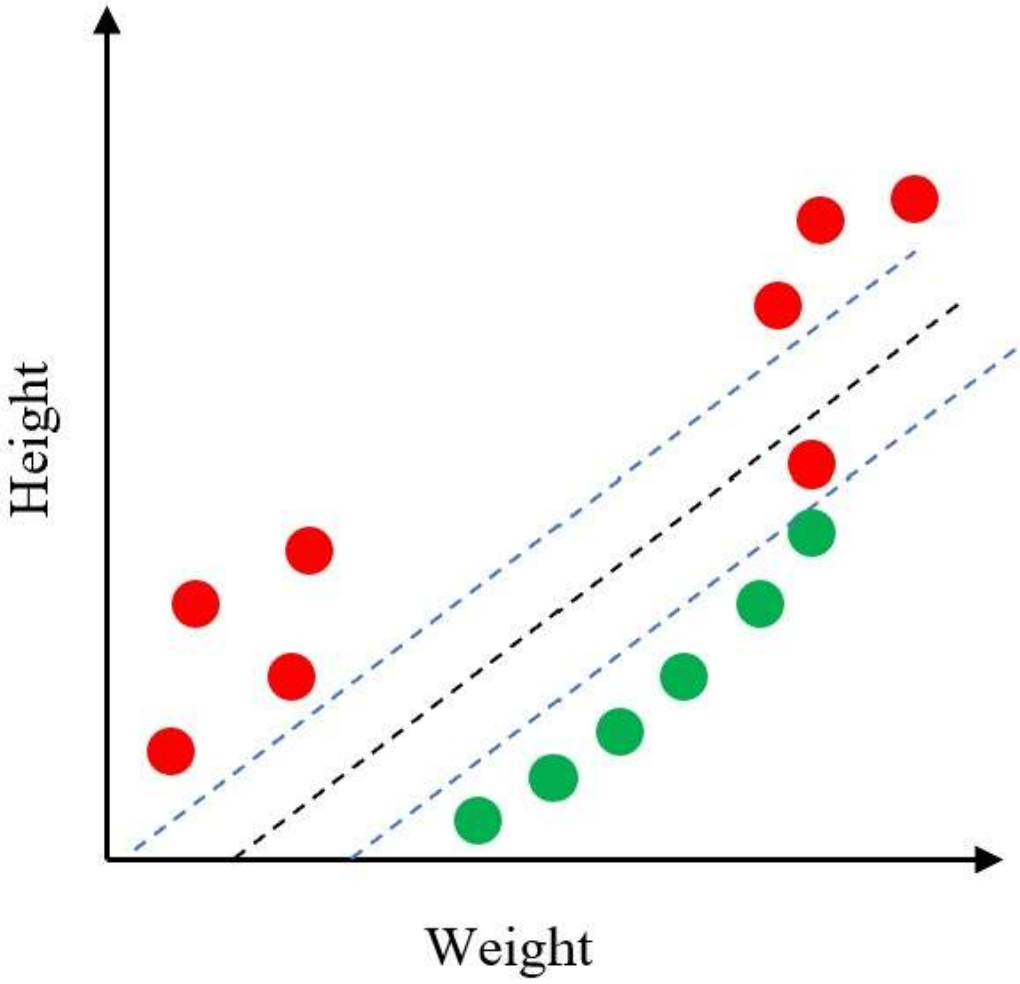


Figure 7.06. Soft margins upon a 2-D plane by Arden University.

The data points outside the soft margins are categorised correctly and the data point inside the soft margin is misclassified, but as we defined earlier on, this misclassification is ok in the long run. Having explained SVM in the manner in which we have, we can now begin to move into more complex methodologies: in other words, adding a third dimension. When we begin moving into the environments of 3rd and higher dimensional planes, the SVC transforms from a line to a plane. Not easily represented here, but it would look something similar to Figure 7.07 below, with the inclusion of the plane in yellow.

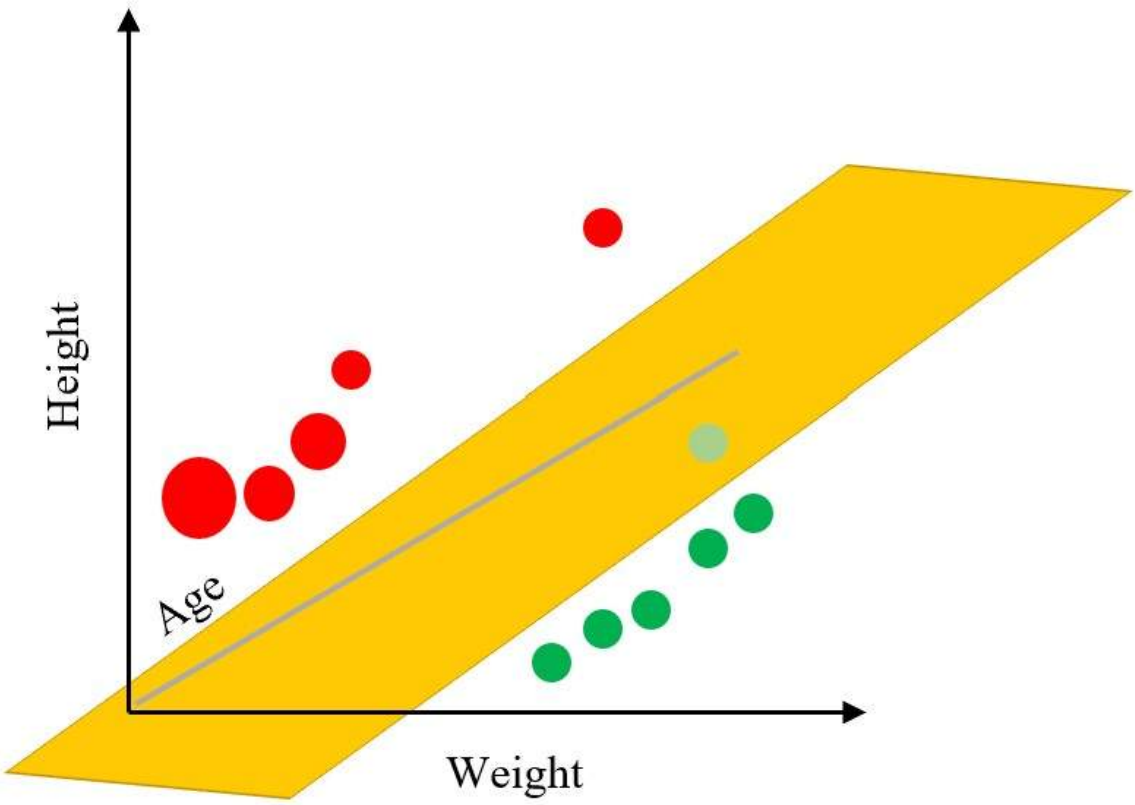


Figure 7.07. Data represented within a 3-D plane referred to as a hyperplane by Arden University.

The advantageous thing about SVC is that we can keep adding dimensions to its complexity and when this occurs, the SVC is commonly referred to as a ***Hyperplane***. However, human beings can not visually represent any dimensions higher than three, so we will not be visually representing anything above 3-D here. In Figure 7.07, if new data points were added, then anything above the plane would be classed as over and anything under the plane would be classed as ideal. Pretty cool!

There is one more item that we need to demonstrate, which will bring us to the tile of this lesson, i.e., SMV. Say that the dataset we have now does not look like the dog data we have been using. Let's say for example that a medicine test was conducted on sixteen patients. The results show in Figure 7.08 that the medicine only works when the correct amount is given.

Figure 7.08. Alternate dataset representation in 1-D by Arden University.

When we are presented with a dataset as shown in Figure 7.08, it really does not matter where the threshold is placed, it will always give a misclassification: this is where SVC breaks down, as it does not like, or can handle datasets of this ilk. So what do we do from this point, forget the study and try something easier? Of course not, and this is where SVM come into play.

To use SVM we need to transform the data points using what is known as a Kernel function and the Kernel function will allow us to square each data point. Let us break this down even further. If we begin a study with a 1-D dataset, we use the Kernel function to transform the data into a 2-D dataset, we can then apply the SVC to the data in order to be able to classify data in higher dimensions. Figure 7.09 will graphically represent the development from a 1-D dataset to 2-D, by squaring each data point (Kernel function).

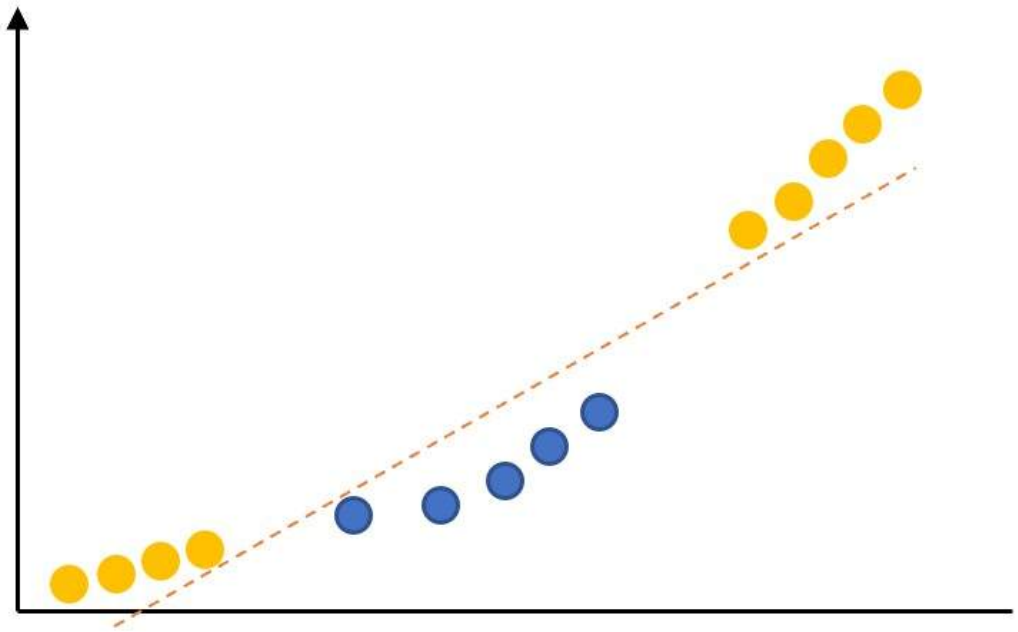


Figure 7.09. Development from a 1-D plane to a 2-D plane by Arden University.

The question I am sure you are asking yourselves at the moment is, “Why are the data points square by the Kernel function, why not some other transformation”? If you are asking yourselves this, then this is a good question and warrants a further explanation. As we have transformed the data from 1-D to 2-D, we square the data points to be able to determine the SVC. If we had a 2-D dataset, we would cube all data points to move to a higher 3rd dimension and so on. When moving to a higher dimension, the Kernel function is represented as d. We then use cross validation to find the optimum SVC. This is one of the key fundamentals with SVM, we always, whenever possible, move the data into a higher dimension so that we can calculate, using misclassification and cross validation to determine the SVC. That’s basically it.

If we had dived straight in with hyperplanes and higher dimensional models, it would have seemed very confusing, which is why we have explained the full process of SVM from ground principles up. Hopefully you now have a firm grip on SVM and why they are so widely used for classification purposes.

Continue

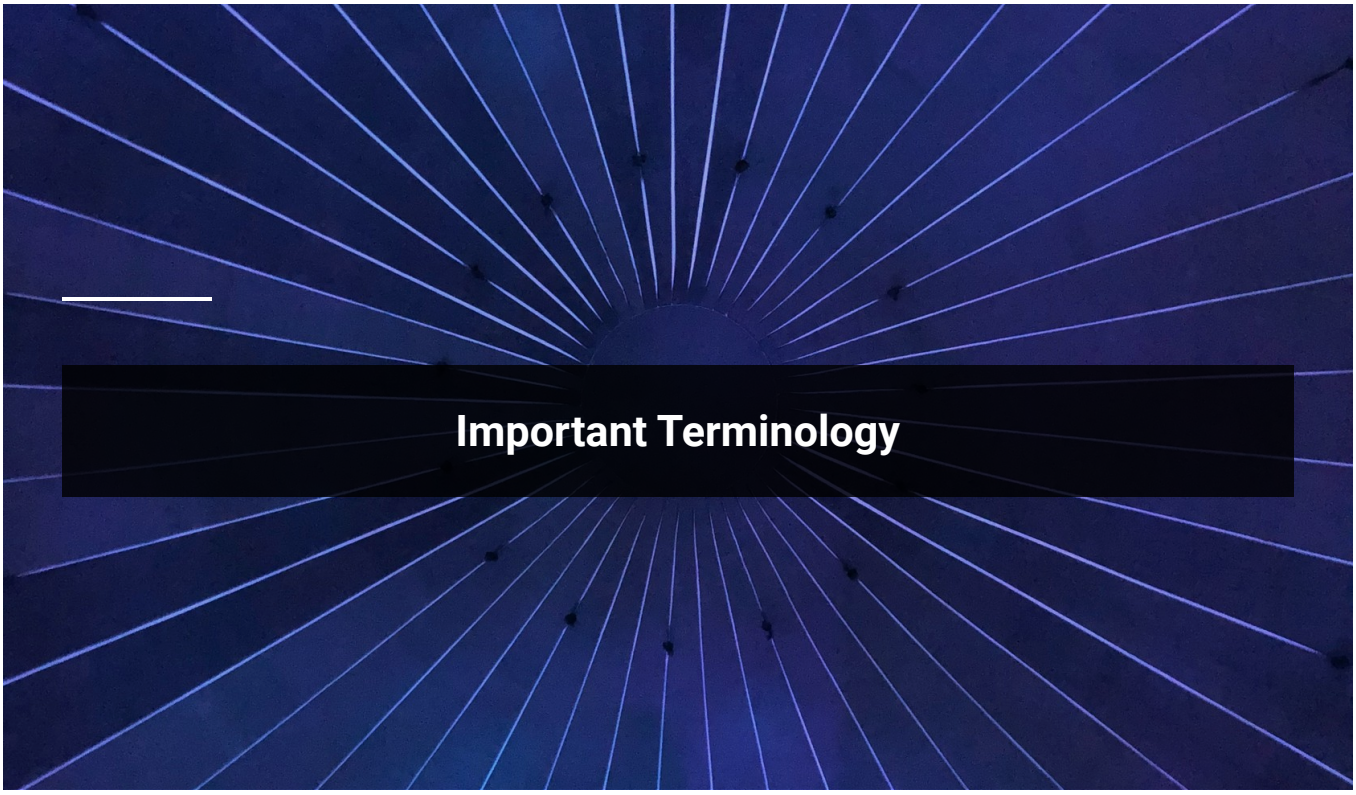


Image from Articulate Rise 360 Content Library.

Kernel: A Kernel is a function for converting lower dimensional datasets into higher dimension ones, while searching for the optimum hyperplane, thus reducing computational costs.

Hyperplane: An optimal separating line (1-D), plane (2-D) between data classes.

Boundary line: Apart from a hyperplane, there are two more lines within an SVM study, namely margins. These margins are the boundary lines and the different classes within a classification study are separated by these margins.

Continue

Topic 1 - Activity 1

Discussion on iLearn - SVM

In your own words describe what SVM is and what it is used for. Also provide some real-world environments where the application would be most suited. Disclose your real-world application within the group's forum for further discussion.

Continue

Topic 1 - Activity 2

Reading

- Chapter 1: Introduction (pp. 11-31) from Introduction to Machine Learning with Python: A Guide for Data Scientists by Müller and Guido.
- Chapter 1: The IDLE Integrated Development Learning Environment (pp. 23-40) from Python for Scientific Computing and Artificial Intelligence by Lynch.
- Chapter 1: Preliminaries (pp. 12-30) from Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter by McKinney.

- Chapter 2: Python Programming Primer (pp. 71-136) from Statistics and Data Visualisation with Python by Rogel-Salazar.

Continue

TOPIC 1 SUMMARY

Well done, you have completed Topic 1.

Within this topic we have given the ideology of SVM and given graphical representation too. SVM is not only limited to 2-D and as we have seen as we have progressed through the topic, SVM can be applied in multi-dimensions, although when higher dimensions are sort, the process can become quite complex, quite quickly. However, now that we have a solid, rounded understanding of what SVM is, we can now actually create a full investigation using the algorithm in Topic 2.

See you there.

Continue to Topic 2

Topic 2: Building an SVM Model

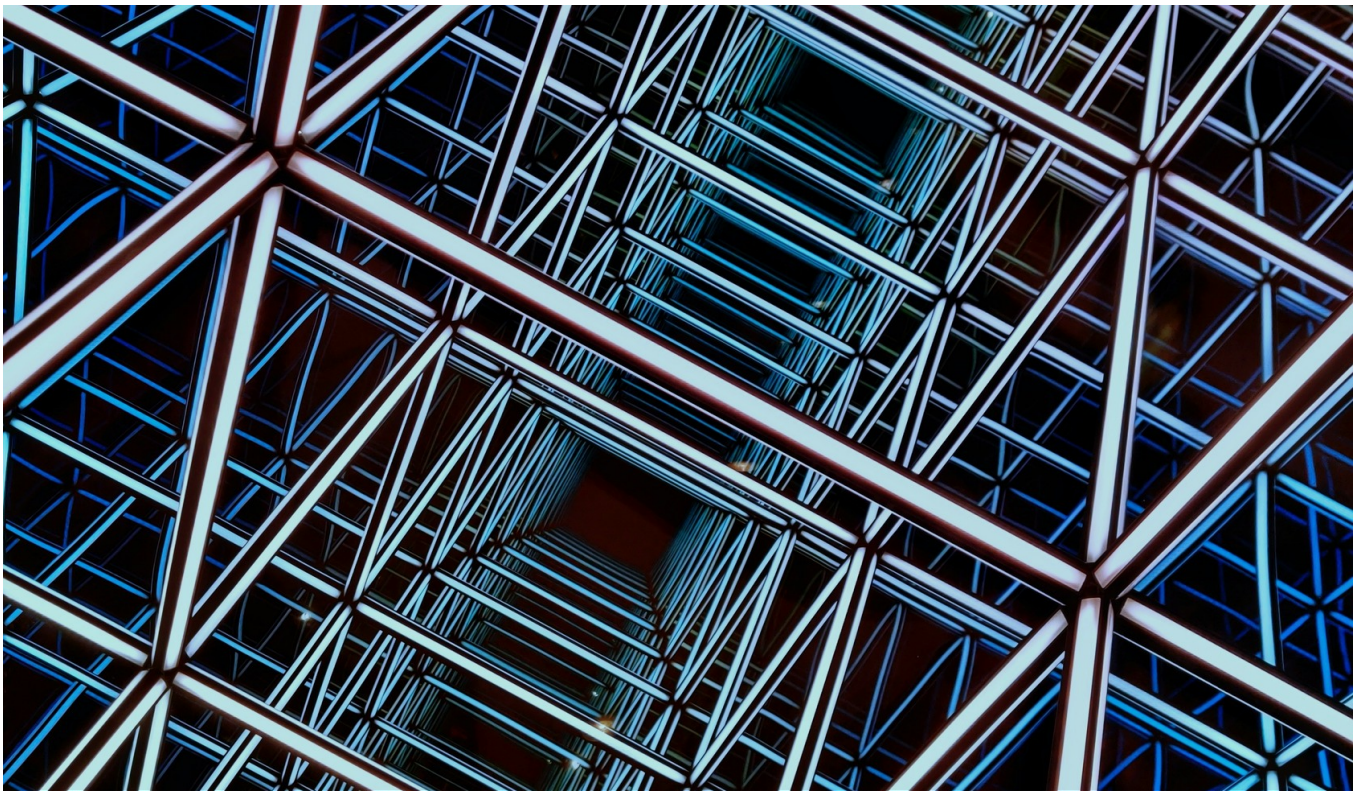


Image from Articulate Rise 360 Content Library.

Having now given a detailed explanation of what SVM is from foundational principles up and more importantly why it is used, the next logical step would be to give an example on how to build one in Python and evaluate.

As with all examples within this module, a detailed step-by-step approach is given via comments. In doing so, it will give the new analyst a clear path and methodology in building one for themselves. In the example given below, a random dataset of 100 points is created and then an SVM model, using a polynomial Kernel. When this particular Kernel is included and results visually represented, an alternative, Linear Kernel is applied, hopefully to improve the accuracy of the classification model.

Continue

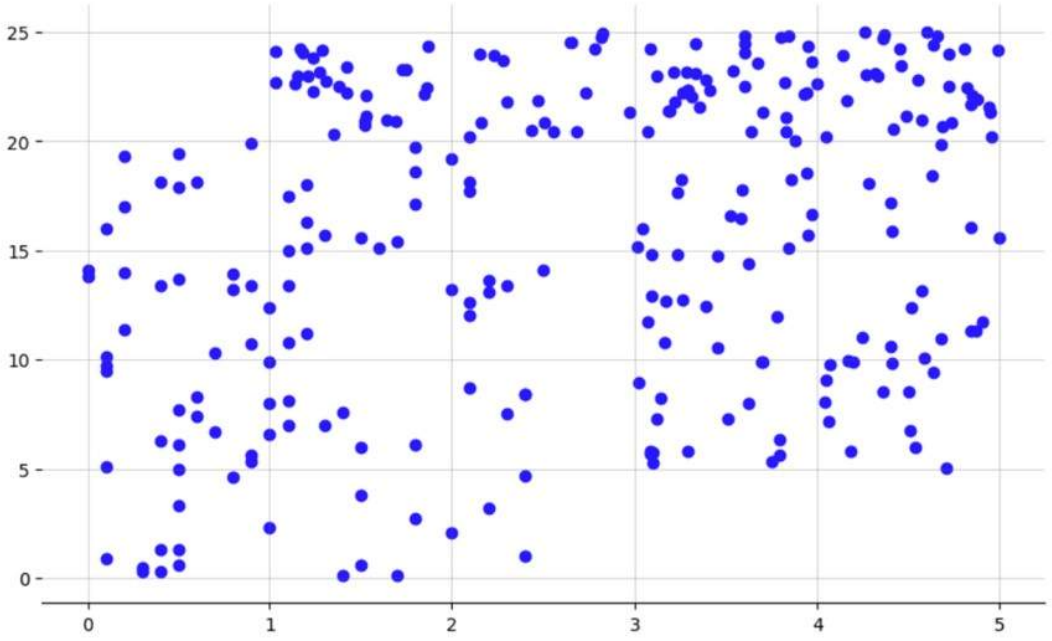
Example

```
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
def generate_random_dataset(size):
    x = []
    y = []
    target = []
    for i in range(size):
        # Defining class zero
        x.append(np.round(random.uniform(0, 2.5), 1))
        y.append(np.round(random.uniform(0, 20), 1))
        target.append(0)
        # Defining class one
        x.append(np.round(random.uniform(1, 5), 2))
        y.append(np.round(random.uniform(20, 25), 2))
        target.append(1)
        x.append(np.round(random.uniform(3, 5), 2))
        y.append(np.round(random.uniform(5, 25), 2))
        target.append(1)
    df_x = pd.DataFrame(data=x)
    df_y = pd.DataFrame(data=y)
    df_target = pd.DataFrame(data=target)
    data_frame = pd.concat([df_x, df_y], ignore_index=True, axis=1)
    data_frame = pd.concat([data_frame, df_target], ignore_index=True, axis=1)
    data_frame.columns = ['x', 'y', 'target']
    return data_frame
```

Figure 7.10a. Resulting SVM investigation based upon the example shown in Google Colab by Arden University. (Accessible code).

```
# Generate a dataset
size = 100
dataset = generate_random_dataset(size)
features = dataset[['x', 'y']]
label = dataset['target']
# Setting 30% of the dataset for training
test_size = int(np.round(size * 0.3, 0))
# Splitting the  dataset into training and testing sets
x_train = features[:-test_size].values
y_train = label[:-test_size].values
x_test = features[-test_size:].values
y_test = label[-test_size:].values
# Plotting the training set
fig, ax = plt.subplots(figsize=(10, 6))
ax.spines['top'].set_visible(False)
ax.spines['left'].set_visible(False)
ax.spines['right'].set_visible(False)
# Generating gridlines
ax.grid(color='grey', linestyle='-', linewidth=0.50, alpha=0.5)
ax.scatter(features[:-test_size]['x'], features[:-test_size]['y'], color="blue")
plt.show()
```

Figure 7.10b. Resulting SVM investigation based upon the example shown in Google Colab by Arden University. (Accessible code).




```
from sklearn import svm
model = svm.SVC(kernel='poly', degree=2)
model.fit(x_train, y_train)
```

```
SVC
SVC(degree=2, kernel='poly')
```

Figure 7.10c. Resulting SVM investigation based upon the example shown in Google Colab by Arden University. (Accessible code).

```
fig, ax = plt.subplots(figsize=(10, 6))
# Removing the right border
ax.spines['top'].set_visible(False)
ax.spines['left'].set_visible(False)
ax.spines['right'].set_visible(False)
# Create a grid to evaluate model
xx = np.linspace(-1, max(features['x']) + 1, len(x_train))
yy = np.linspace(0, max(features['y']) + 1, len(y_train))
YY, XX = np.meshgrid(yy, xx)
xy = np.vstack([XX.ravel(), YY.ravel()]).T
train_size = len(features[:-test_size]['x'])
# Assigning different colours to the classes
colors = y_train
colors = np.where(colors == 1, 'red', 'green')
# Plotting the dataset
ax.scatter(features[:-test_size]['x'], features[:-test_size]['y'], c=colors)
# Calculating the separating hyperplane
Z = model.decision_function(xy).reshape(XX.shape)
# Detailing a decision boundary and margins
ax.contour(XX, YY, Z, colors='k', levels=[-1, 0, 1], alpha=0.5, linestyles=['--', '-', '--'])
# Highlighting support vectors with a circle
ax.scatter(model.support_vectors_[:, 0], model.support_vectors_[:, 1], s=100, linewidth=1, facecolors='none', edgecolors='k')
plt.show()
```

Figure 7.10d. Resulting SVM investigation based upon the example shown in Google Colab by Arden University. (Accessible code).

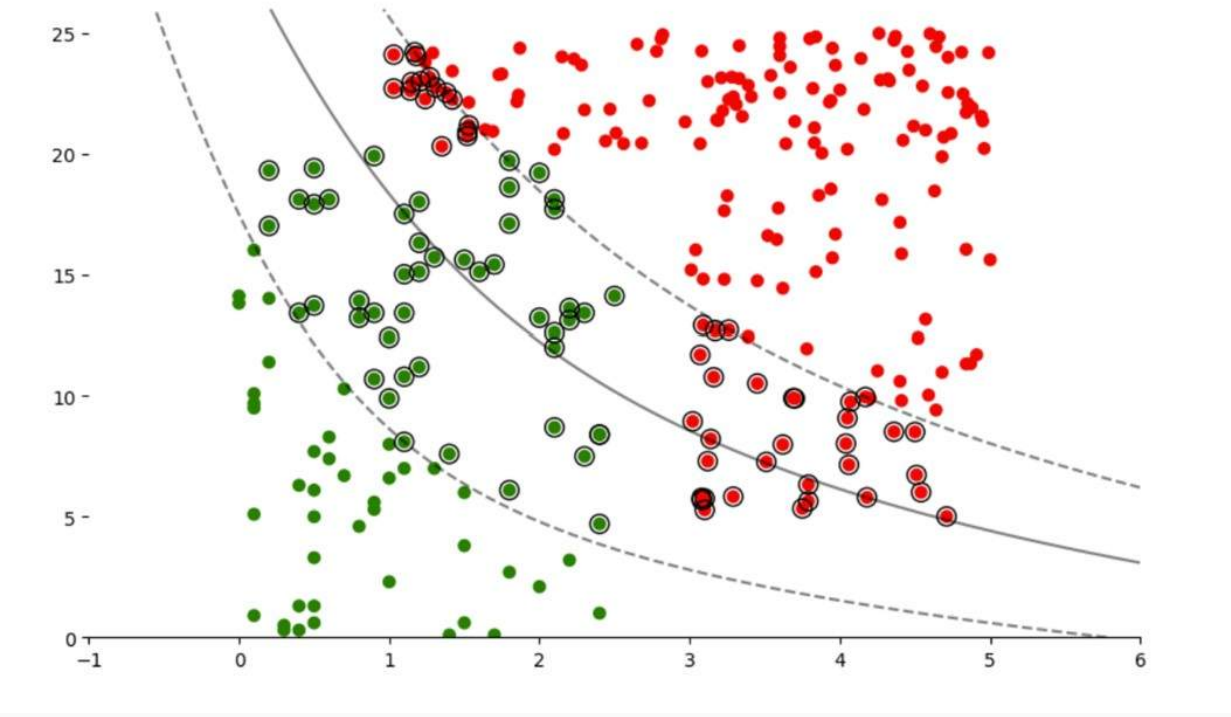


Figure 7.10e. Resulting SVM investigation based upon the example shown in Google Colab by Arden University.

```
[31] from sklearn.metrics import accuracy_score
     predictions_poly = model.predict(x_test)
     accuracy_poly = accuracy_score(y_test, predictions_poly)
     print("2nd degree polynomial Kernel\nAccuracy (normalized): " + str(accuracy_poly))

     2nd degree polynomial Kernel
     Accuracy (normalized): 0.9
```

```
     model = svm.SVC(kernel='linear')
     model.fit(x_train, y_train)
```

```
     SVC
     SVC(kernel='linear')
```

Figure 7.10f. Resulting SVM investigation based upon the example shown in Google Colab by Arden University. (Accessible code).

```
fig, ax = plt.subplots(figsize=(10, 6))
# Remove right border
ax.spines['top'].set_visible(False)
ax.spines['left'].set_visible(False)
ax.spines['right'].set_visible(False)
# Creating a grid to evaluate model
xx = np.linspace(-1, max(features['x']) + 1, len(x_train))
yy = np.linspace(0, max(features['y']) + 1, len(y_train))
YY, XX = np.meshgrid(yy, xx)
xy = np.vstack([XX.ravel(), YY.ravel()]).T
train_size = len(features[:-test_size]['x'])
# Assigning different colours to classes
colors = y_train
colors = np.where(colors == 1, 'red', 'green')
# Plotting the dataset
ax.scatter(features[:-test_size]['x'], features[:-test_size]['y'], c=colors)
# Defining hyperplane
Z = model.decision_function(xy).reshape(XX.shape)
# Including decision boundary and margins
ax.contour(XX, YY, Z, colors='k', levels=[-1, 0, 1], alpha=0.5, linestyles=['--', '-', '--'])
# Highlighting support vectors with a circle
ax.scatter(model.support_vectors_[:, 0], model.support_vectors_[:, 1], s=100, linewidth=1, facecolors='none', edgecolors='k')
plt.show()
```

Figure 7.10g. Resulting SVM investigation based upon the example shown in Google Colab by Arden University. (Accessible code)

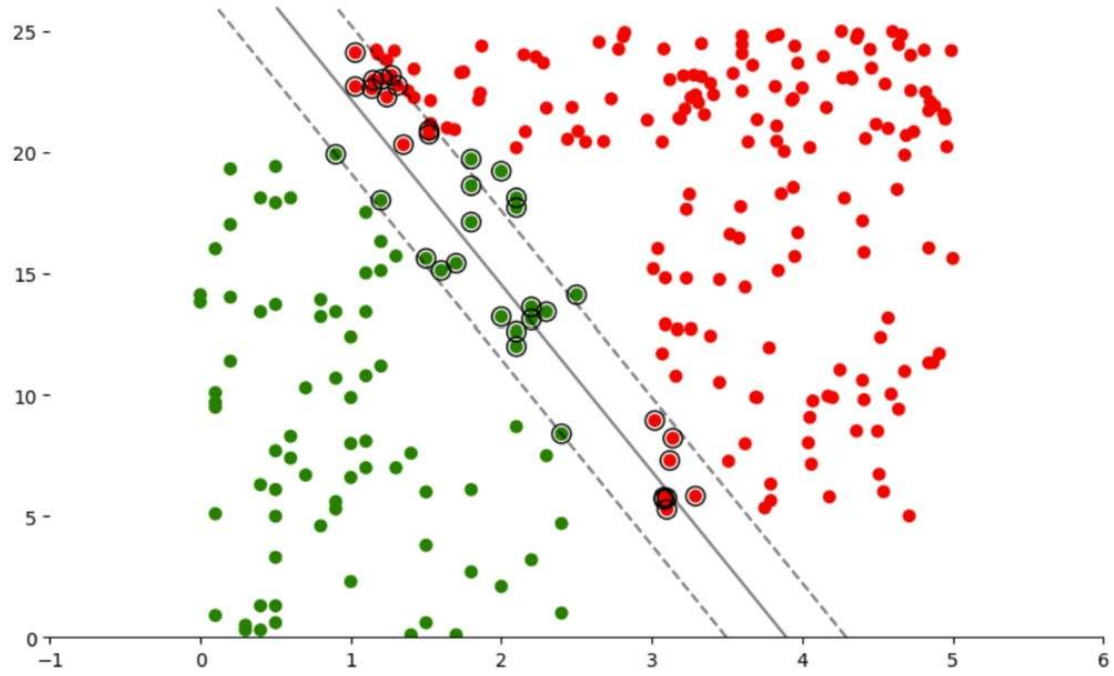


Figure 7.10h. Resulting SVM investigation based upon the example shown in Google Colab by Arden University.

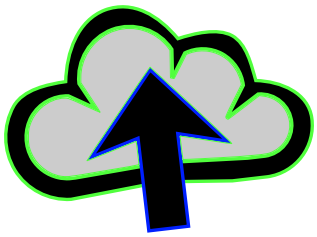
Continue

Topic 2 - Activity 1

Discussion on iLearn

Having now run through the detailed example within this topic, take your time to change and alter variables, including their quantities. Can you improve the model even further by doing so? Post your results in the groups' discussion forum.

Import any open source dataset and perform an SVM study, following the approach detailed within the lesson. Disclose your results as a mini report within the discussion forum to promote even further investigations and discussions with the tutor and peers.



Continue

Topic 2 - Activity 1

Reading

- Chapter 2: Supervised Learning (pp. 33-102) from Introduction to Machine Learning with Python: A Guide for Data Scientists by Müller and Guido.
- Chapter 2: Anaconda, Spyder and the Libraries NumPy, Matplotlib and SymPy (pp. 41-52) from Python for Scientific Computing and Artificial Intelligence by Lynch.
- Chapter 2: Python Language Basics, IPython, and Jupyter Notebooks (pp. 30-56) from Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter by McKinney.
- Chapter 3: Snakes, Bears & Other Numerical Beasts: NumPy, SciPy & pandas (pp. 137-178) from Statistics and Data Visualisation with Python by Rogel-Salazar.

Continue

TOPIC 2 SUMMARY

Well done, you have completed Topic 2.

Topic 2 developed on from the theory given within Topic 1. Here we have given a full SVM investigation. As you have no doubt determined, an SVM study is quite involved, yet it is important that you understand the steps taken to conduct such an investigation as they are used frequently within the real world. If you found the topic challenging, then this is perfectly normal and I encourage you to look back at the example as many times as needed.

SVM now complete, we can move on to SVR in Topic 3.

See you there.

Continue to Topic 3

Topic 3: Support Vector Regression

Image from Articulate Rise 360 Content Library.

As this topic is closely related to regression, it would be advantageous for us to remind ourselves how a regression line within such an analysis is derived. The regression line is created using the Ordinary Least Squares formula, as shown below.

$$\textit{Regression Line} = (y - \hat{y})^2 \text{ --> min}$$

Basically, we are trying to reduce the errors that will occur within such a study, therefore ensuring that the regression analysis is as accurate as possible, for time series predictions. The errors, in essence, are how far away each data point is from the regression line and analysts strive to reduce them as much as possible without compromising the overall dataset, as Figure 7.11 demonstrates.

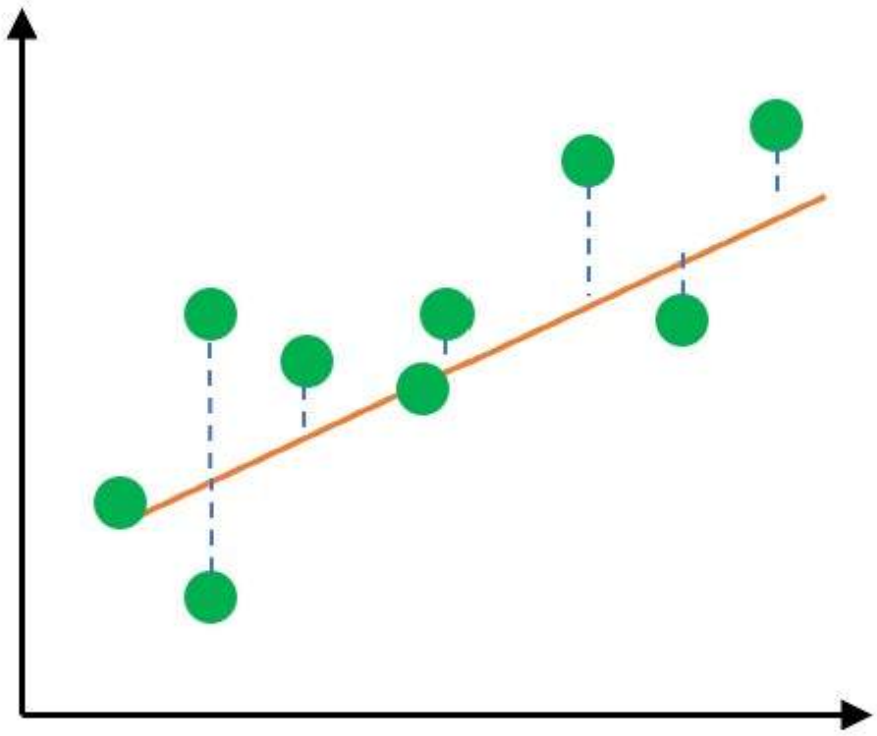


Figure 7.11. Regular regression model with errors by Arden University.

Having now reminded ourselves with simple linear regression and how the regression line is calculated and why, there is a development on from this methodology which can help the analysis even further in model accuracy and this is the methos known as Support Vector Regression (SVR).

The SVR methodology is very similar to simple linear regression, but with one additional application, a sleeve or cylinder, imposed around the regression line, as demonstrated in Figure 7.12.

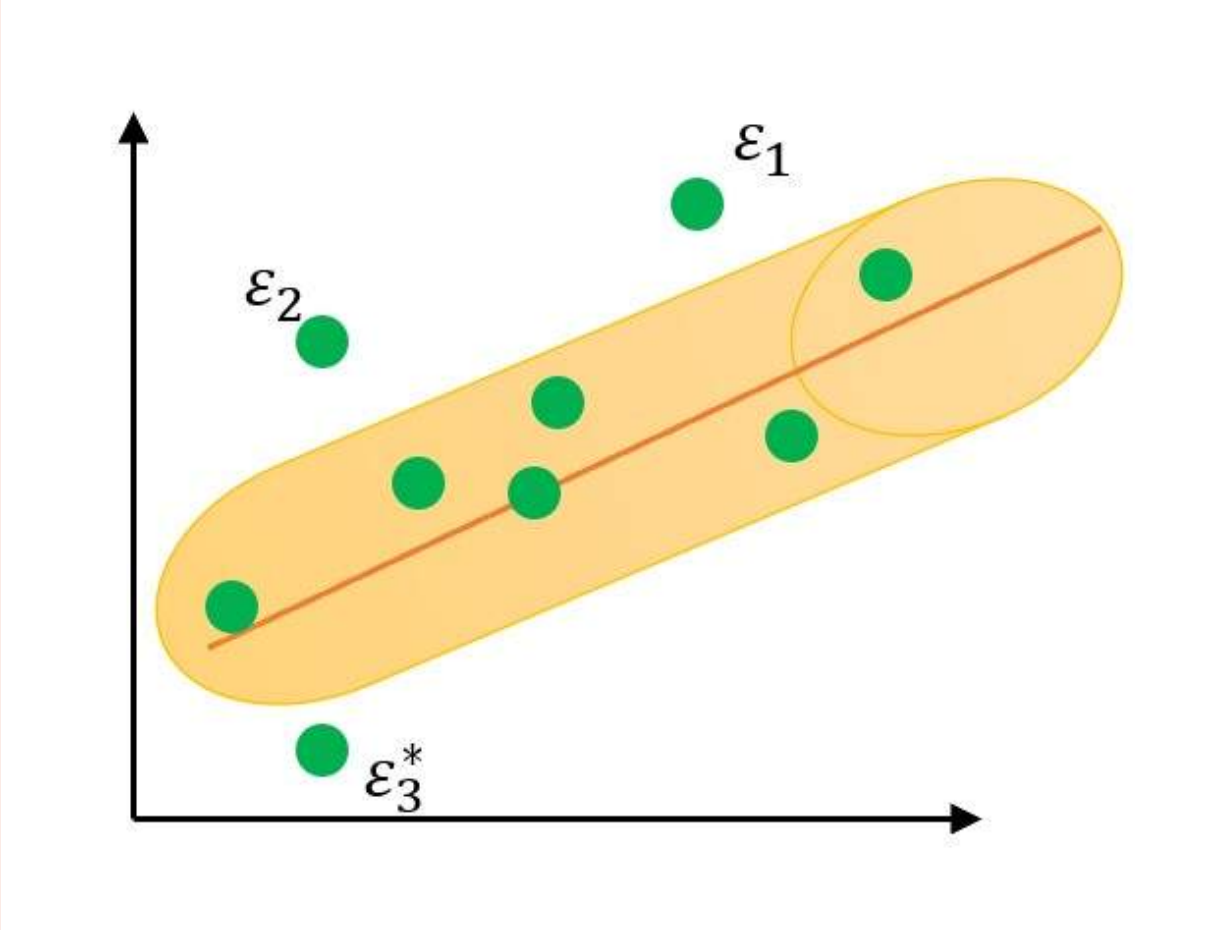


Figure 7.12. Soft margins applied as a sleeve for SVR by Arden University.

The cylinder's formal mathematical term is the *ε insensitive tube*. Basically, as the simple linear regression analysis takes into account ALL data points, SVR does not take into account the errors that are located within the imposed cylinder. The only errors that are considered are the ones that are outside of the cylinder. In other words, we are allowing the model to have a specific margin of error, as shown in Figure 7.12, denoted as *ε*.

The data points which are outside of the cylinder, i.e., the errors, are all calculated from the error itself to the closest cylinder boundary, NOT the regression line. The errors, formally denoted as *slack variables* are defined as ε_i if above the regression line, or ε_i^* if below the regression line. The reason why this analysing technique is called SVR is because if we take any data point within the regression analysis, we can impose a straight line from the origin and these straight lines, are be represented as vectors, or support vectors as Figure 7.13 demonstrates.

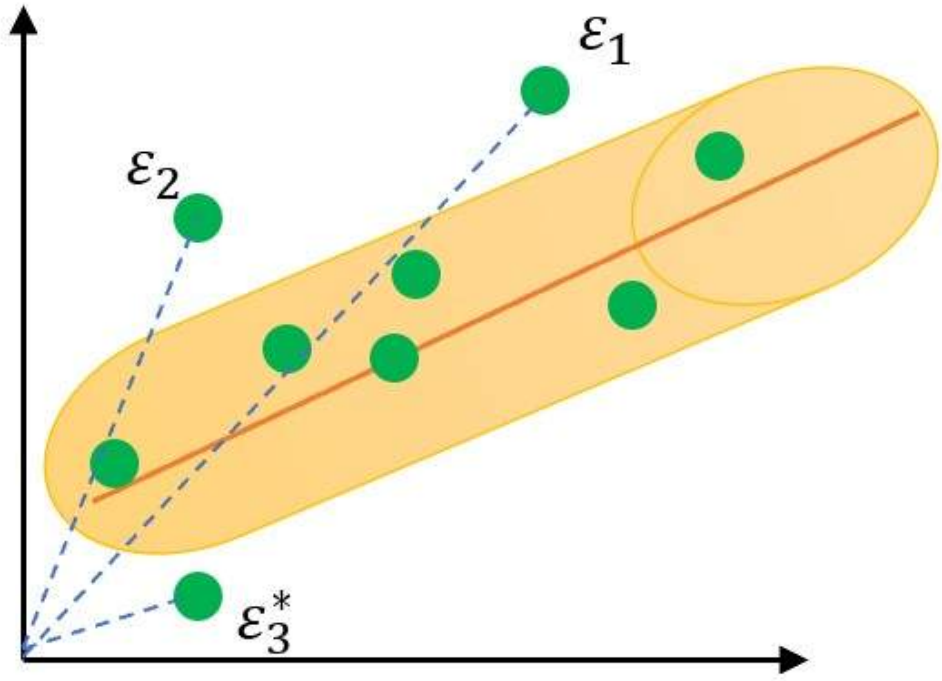


Figure 7.13. Demonstrating the errors within an SVR by Arden University.

We shall now run through a full example on a self-generated, non-linear function. Two differing Kernels will be applied, plus tested, as well as a baseline regression test. In approaching the example in this manner, you can hopefully see how a SVR model can improve the accuracy of the model, if at all. As always, the coded screen shots will be annotated at each point so that it is easier to follow throughout.

Continue

Example

Importing initial relevant libraries

```
[201] import numpy as np
      import pandas as pd
      import matplotlib.pyplot as plt
```

Generating a none linear function to use as our SVR example

```
[202] def nonlinear(array):
          return (10*array[:,0]-np.exp(0.01*array[:,1]+np.log(1+array[:,2]**2)))/(array[:,3]**2+4)
```

Figure 7.14a. A full coded example of Support Vector Regression in Google Colab by Arden University. (Accessible code).

Generating features and data for regression

```
n_samples = 100
n_features = 5
```

```
[204] x = 5*np.random.rand(n_samples,n_features)
```

```
[205] y = nonlinear(x)+np.random.randn(n_samples)
```

```
[206] y=y.reshape(n_samples,1)
```

```
[207] df = pd.DataFrame(data=np.hstack((x,y)),columns=['X1','X2','X3','X4','X5','y'])
```

```
[208] df.head()
```

Plotting the data

```
fig,ax = plt.subplots(2,2,figsize=(8,6))
ax = ax.ravel()
for i in range(4):
    ax[i].scatter(df[df.columns[i]],df['y'],edgecolor='k',color='blue',alpha=0.50)
    ax[i].set_title(f"{df.columns[i]} against Y",fontsize=12)
    ax[i].grid(True)
plt.show()
```

Figure 7.14b. A full coded example of Support Vector Regression in Google Colab by Arden University. (Accessible code).

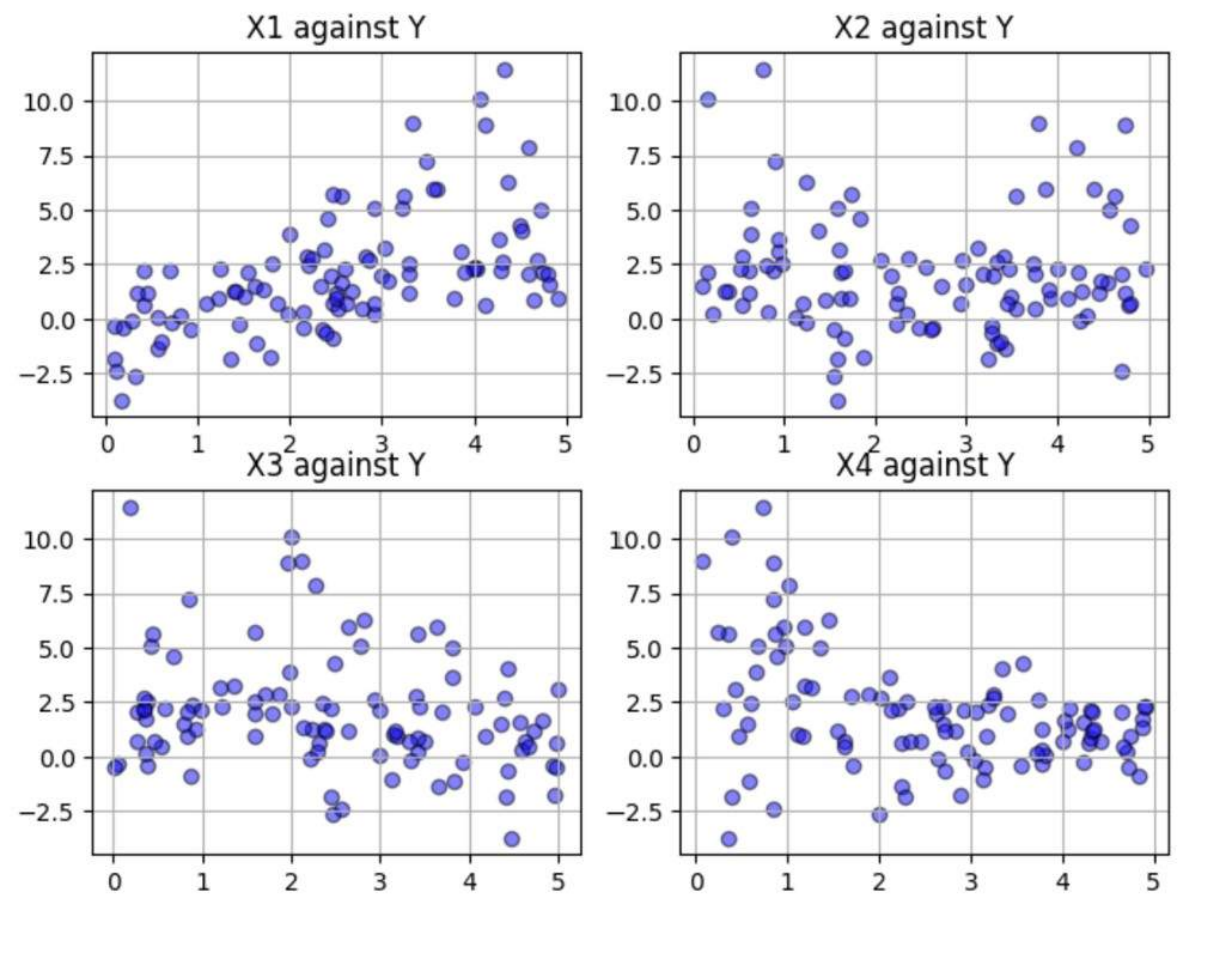


Figure 7.14c. A full coded example of Support Vector Regression in Google Colab by Arden University.

Splitting the data set into Train and Test

```
[210] X = df[['X1','X2','X3','X4','X5']]
      y = df['y']
```

```
[211] from sklearn.model_selection import train_test_split
```

```
[212] X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)
```

SVR with applied Linear Kernal

Here is scikit-learn's SVR doc: https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVR.html

```
[213] from sklearn.svm import SVR
      svr_linear = SVR(kernel='linear',gamma='scale', C=2.0, epsilon=0.5)
      svr_linear.fit(X_train, y_train)
```

```
SVR
SVR(C=2.0, epsilon=0.5, kernel='linear')
```

Figure 7.14d. A full coded example of Support Vector Regression in Google Colab by Arden University. (Accessible code).

Initial Test Score

```
[214] svr_linear.score(X_test,y_test)
```

```
0.40373793733140806
```

Using Linear Regression as a baseline

```
from sklearn.linear_model import LinearRegression
linear = LinearRegression()
linear.fit(X_train,y_train)
```

```
▾ LinearRegression
LinearRegression()
```

Baseline score

```
[216] linear.score(X_test,y_test)
```

```
0.4401706773406069
```

Figure 7.14e. A full coded example of Support Vector Regression in Google Colab by Arden University. (Accessible code).

SVR with applied Gaussian Kernal

```
svr_rbf = SVR(kernel='rbf',gamma='scale', C=2.0, epsilon=0.5)
svr_rbf.fit(X_train, y_train)
```

```
SVR
SVR(C=2.0, epsilon=0.5)
```

```
[218] svr_rbf.score(X_test,y_test)
```

```
0.5682476261719616
```

```
[219] from sklearn.metrics import mean_squared_error
```

```
[220] print("linear SVR:",np.sqrt(mean_squared_error(y_test,svr_linear.predict(X_test))))
      print("kernelized SVR:",np.sqrt(mean_squared_error(y_test,svr_rbf.predict(X_test))))
```

```
linear SVR: 1.7773884759344585
kernelized SVR: 1.5124503890777434
```

Figure 7.14f. A full coded example of Support Vector Regression in Google Colab by Arden University. (Accessible code).

Using hyperparameters to see if the test score be improved further

```
from sklearn.model_selection import GridSearchCV
params = {'C':[0.01,0.05,0.1,0.5,1,2,5],'epsilon':[0.1,0.2,0.5,1]}
grid = GridSearchCV(svr_rbf,param_grid=params,cv=5,scoring='r2',verbose=1,return_train_score=True)
grid.fit(X_train,y_train)
```

```
Fitting 5 folds for each of 28 candidates, totalling 140 fits
▸ GridSearchCV
▸ estimator: SVR
    ▸ SVR
```

Figure 7.14g. A full coded example of Support Vector Regression in Google Colab by Arden University. (Accessible code).

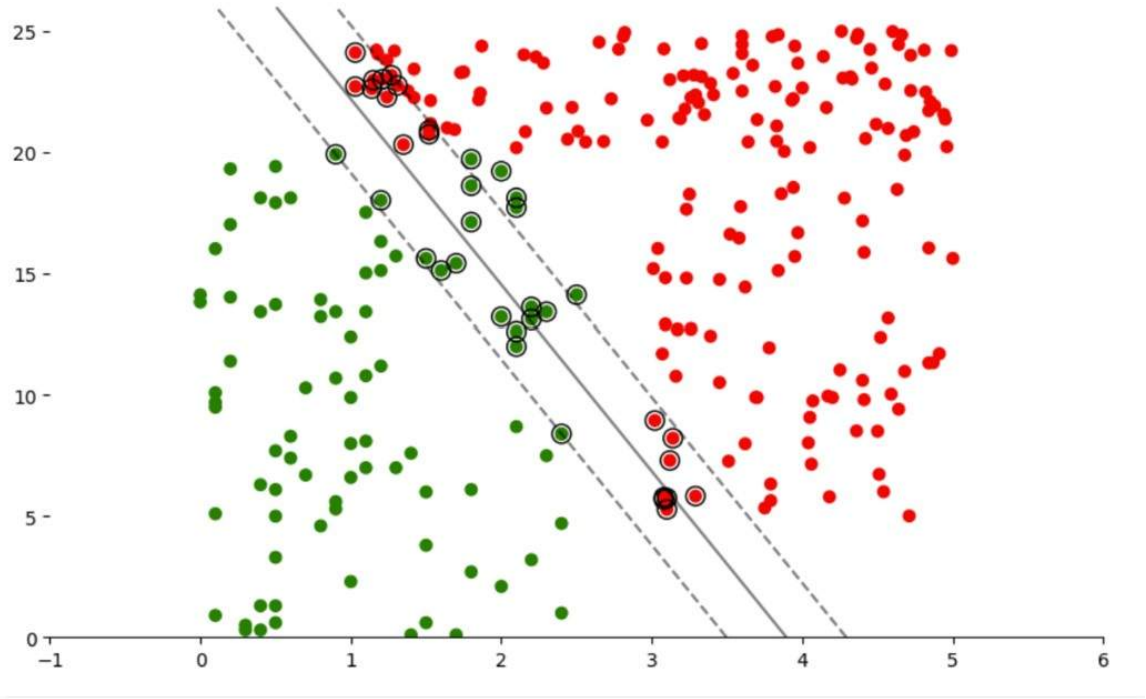


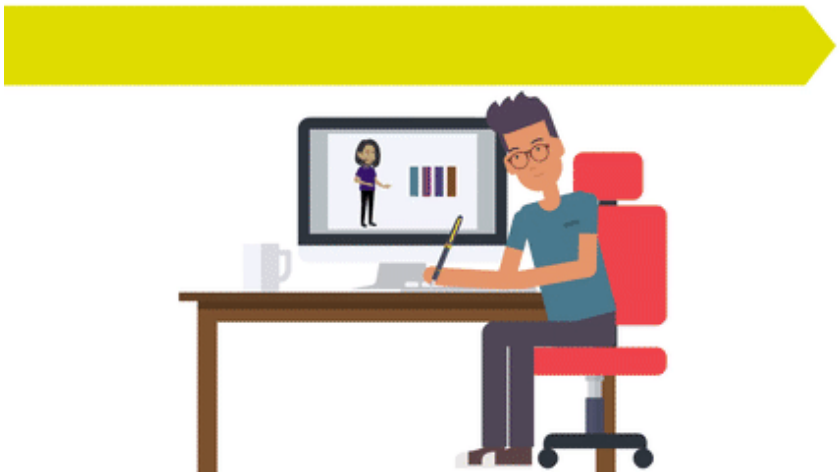
Figure 7.14h. A full coded example of Support Vector Regression in Google Colab by Arden University.

As is clearly evident from the above example, the initial application of the Linear Kernel being applied do not have great results, i.e., 43%. Therefore, to determine whether any improvements could be made a linear regression baseline test was conducted, which resulted in a 44% accuracy, again not great. However, when we investigated the application of the Gaussian Kernel, tested and examined the optimal inputs, we ended with a much-improved model accuracy of 61%. Again, the accuracy, once SVR improvements have been implemented, is not all that much better and if we were in a real-world environment, you would not use this dataset for any further classifications as it would require it to be much higher. However, this was not the point of this example, the point was to see that by applying differing SVR Kernels to the dataset, which was randomly created, could the accuracy of the model be improved, and I think that we have demonstrated that here in this example.

Hurrah!

Topic 3 - Activity 1

Make a Note



In your own words describe what SVR is and what it is used for. Also provide some real-world environments where the application would be most suited.

You can use the below Notepad to help you with this activity, but feel free to use whichever note-taking tool you prefer.

If you decide to use the below Notepad, please remember anything you type in the box below will not be kept when you leave the lesson, so you will need to download your notes for revision and discussion purposes.

My Notes

Input Your Notes Here

Continue

Topic 3 - Activity 2

Reading

- Chapter 3. Unsupervised Learning and Preprocessing (pp. 103-151) from Introduction to Machine Learning with Python: A Guide for Data Scientists by Müller and Guido.
- Chapter 3: Jupyter Notebooks and Google Colab (pp. 53-65) from Python for Scientific Computing and Artificial Intelligence by Lynch.
- Chapter 3. Built-In Data Structures, Functions, and Files (pp. 57-88) from Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter by McKinney.
- Chapter 4: The Measure of All Things – Statistics (pp. 179-216) from Statistics and Data Visualisation with Python by Rogel-Salazar.

Continue

TOPIC 3 SUMMARY

Well done, you have completed Topic 3.

Within this topic we have mainly focused upon the other Support Vector ideology, in other words SVR. Having now completed the topic I hope that you can determine the differences between SVR and SVM and the alternate environments in which they can be applied.

The final topic will sum up what we have learned so far.

See you there.

Continue to Topic 4

Topic 4: Summarising SVM and SVR

Image from Articulate Rise 360 Content Library.

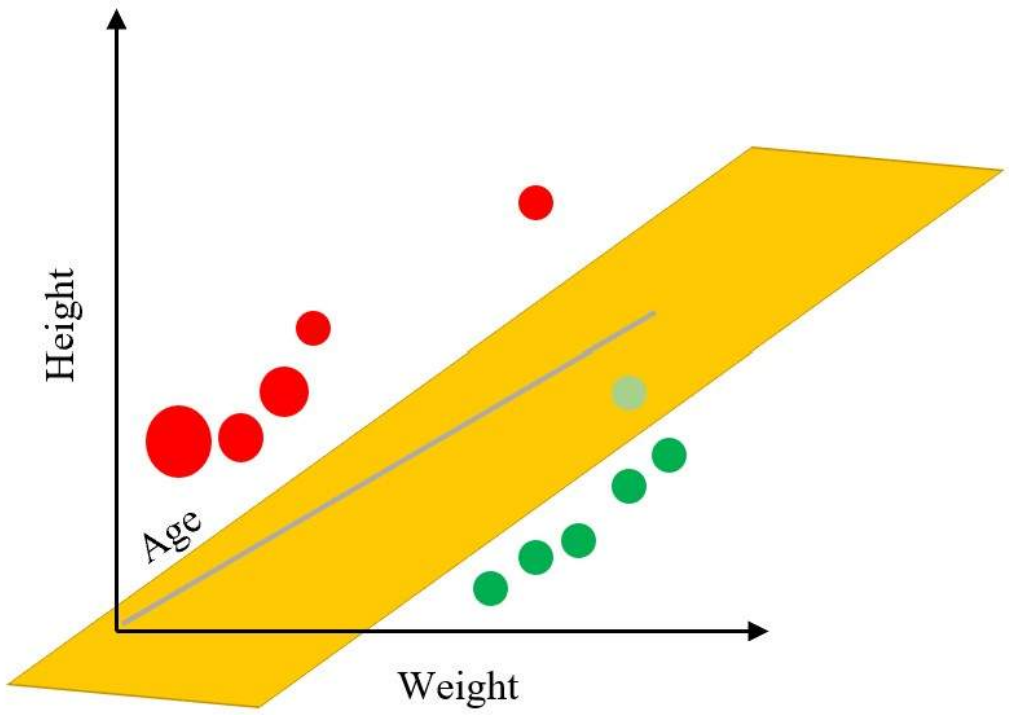


Figure 7.15. Graphical representation of SVM by Arden University. (Accessible code).

As you have no doubt determined, SVM and SVR are quite involved, from a programming perspective. Nevertheless, the techniques are widely used within a machine learning environment for classification purposes.

An SVM algorithm uses supervised learning models to solve, as best as possible, in-depth and complex classification and regression problems, but classifying data points and detecting outliers within the dataset, The technique achieves this by determining boundaries between data points based upon predefined labels.

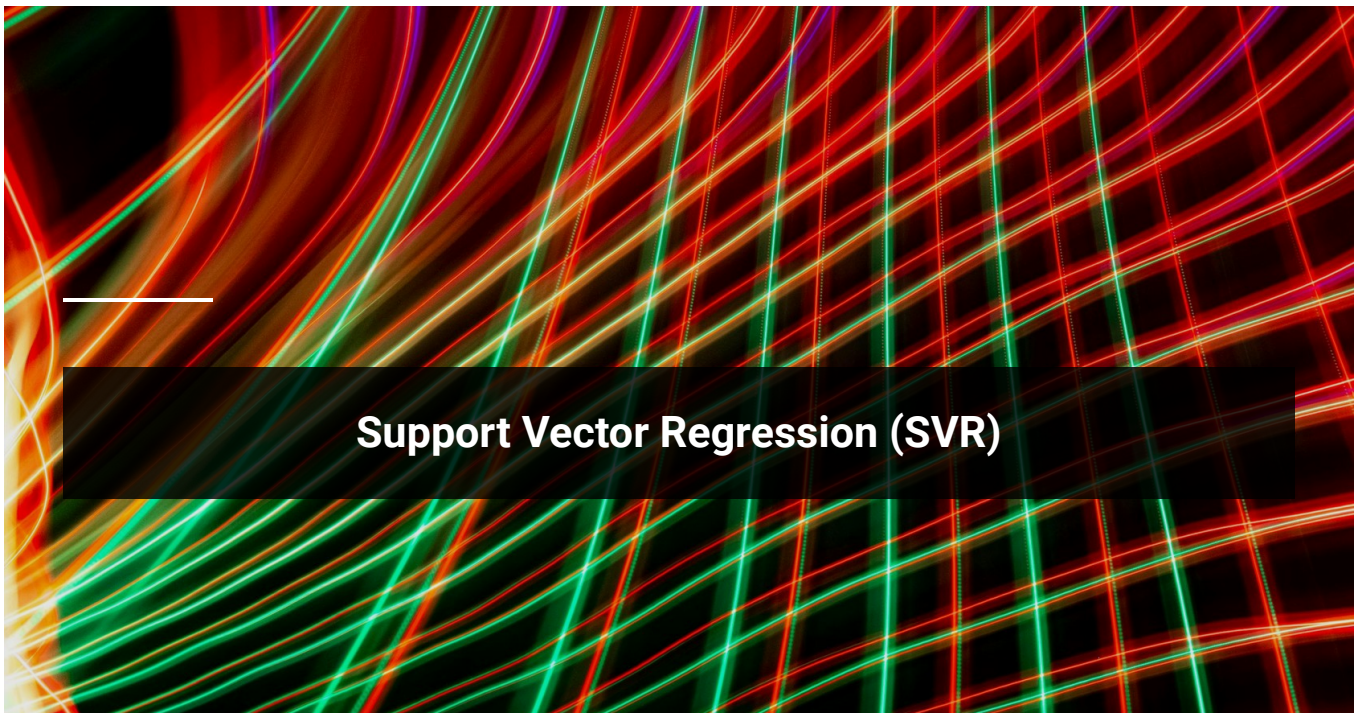
In essence, the main ideal behind SVM and SVR is to determine and employ hyperplanes that aid in the categorisation of singular data points under investigation. Where SVM is solely concerned, a threshold and margins are determined, as we demonstrated in Topic 1. SVM studies are rarely conducted from a single dimension however, usually 2-D and above. When higher dimensions are

involved, the threshold with its associated margins becomes a hyperplane. As the SVM algorithm attempts to maximise the margins between the data points, there is a tendency for misclassification. If, however, the analyst allows for misclassification, then the model does predict more accurately over time.

As we saw in previous topics, SVM uses varying types of Kernels to transform the data. The Kernels do tend to heavily rely on the mapping process of complex datasets for higher dimensions, thus making the data classification easier. This is very useful when applying SVM to real world applications, such as face and expression recognition, text recognition, surface classification and speech recognition.

SVM are crucial to understand and apply for a machine learning analyst when developing applications that will involve predictive models. One of the main reasons why they are so extensively employed is that they are easy to build and easy to understand, even when higher dimensions are explored, i.e., 4th and 5th dimensions.

Continue



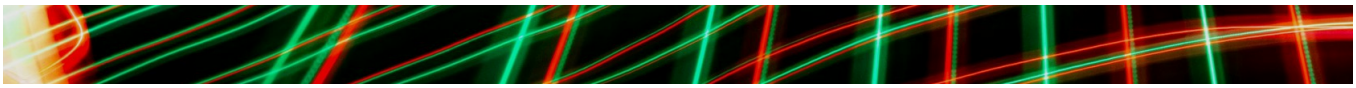


Image from Articulate Rise 360 Content Library.

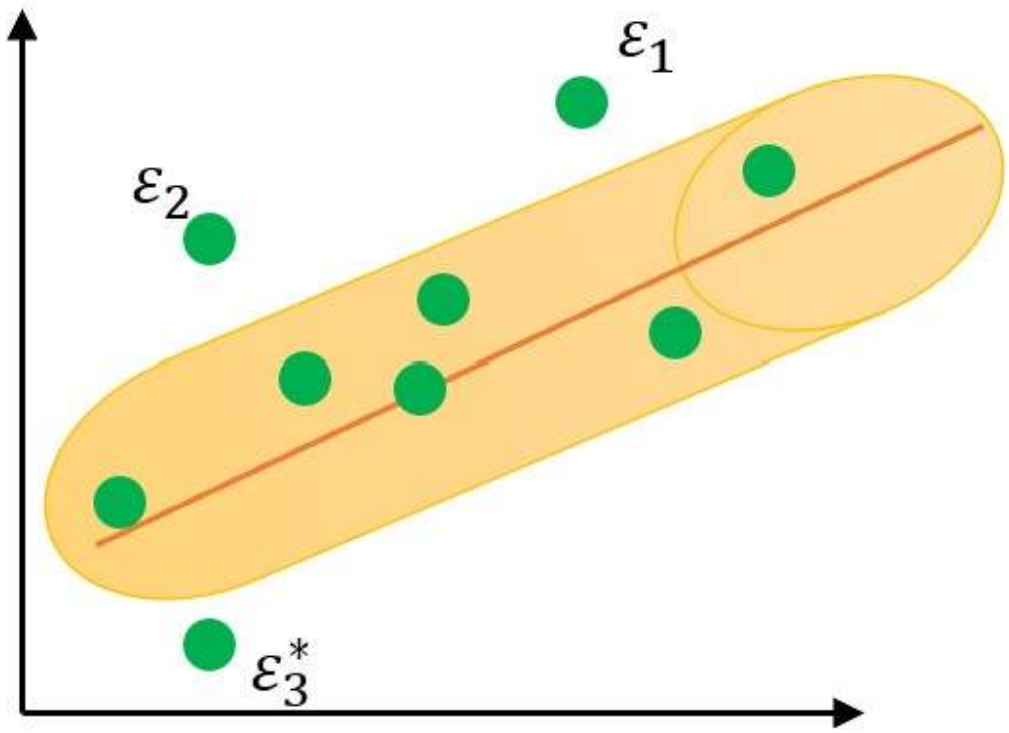


Figure 7.16. Graphical representation of SVR by Arden University.

A clear distinction should be reiterated between SVM and SVR. When SVM is used for classification purposes, it is called Support Vector Classification. When used for regression, it is referred to as Support Vector Regression. As we demonstrated in a previous topic, SVR includes a margin of tolerance, i.e., epsilon. The ε insensitive cylinder/tube is what allows the SVM to become SVR.

Within the insensitive cylinder the optimisation is transformed that best approximates the studies complexities and error. In other words, the cylinder takes into account the data points which are closest to the outer margin lines and only takes into account the data points which are further from this margin. In doing so, it addresses the balance and makes the model much more flexible and accurate.

Continue

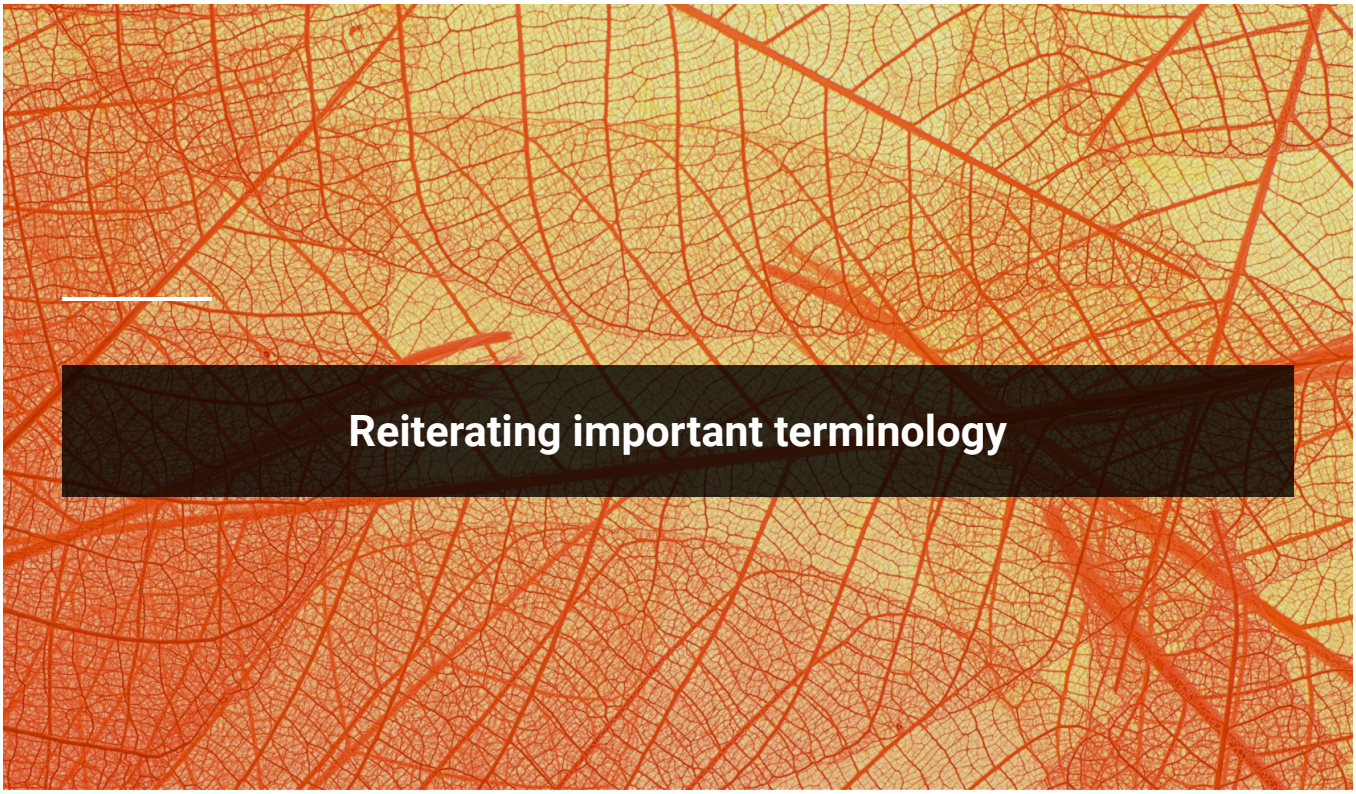


Image from Articulate Rise 360 Content Library.

Kernel: A Kernel is a function for converting lower dimensional datasets into higher dimension ones, while searching for the optimum hyperplane, thus reducing computational costs.

Hyperplane: An optimal separating line (1-D), plane (2-D) between data classes.

Boundary line: Apart from a hyperplane, there are two more lines within an SVM study, namely margins. These margins are the boundary lines and the different classes within a classification study are separated by these margins.

Continue

Topic 4 - Activity 1

Reading

- Chapter 4: Representing Data and Engineering Features (pp. 152-183) from Introduction to Machine Learning with Python: A Guide for Data Scientists by Müller and Guido.
- Chapter 4: Python for AS-Level (High School) Mathematics (pp. 66-84) from Python for Scientific Computing and Artificial Intelligence by Lynch.
- Chapter 4: NumPy Basics: Arrays and Vectorized Computation (pp. 89-121) from Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter by McKinney.
- Chapter 5: Definitely Maybe: Probability and Distributions (pp. 217-304) from Statistics and Data Visualisation with Python by Rogel-Salazar.

Continue

TOPIC 4 SUMMARY

Well done, you have completed Topic 4.

Topic 4 has been concerned with summing up both SVM and SVR. As we mentioned at the very beginning of the lesson, both are very similar, just quite different. Hopefully, now that we have concluded the lesson this has become clear. If not, then you are encouraged to further research the topic to gain a much deeper understanding.

All topics have now been concluded within this lesson.

Continue to Knowledge check

Knowledge check

Select the correct answer for the following quesitons.

1 of 2: What does SVM stand for?

- ◯ Support Vector Machine
- ◯ Supporting Vector Machines
- ◯ Standing Vector Maps

SUBMIT

2 of 2: What does SVR stand for?

- Supporting Vector with Regression
- Standing Vectors with Regression
- Support Vector Regression

SUBMIT

Continue to Lesson summary

Lesson summary

LESSON SUMMARY

Congratulations, you have completed Lesson 7.

Without doubt, full SVM and SVR studies are involved. However, now that you have completed the lesson, you are hopefully much more informed on their processes and where they can be applied.

Within Topic 1 we gave the ideology of SVM, including graphical representation. As we no doubt determined, SVM is not only limited to 2-D and can be applied in multi-dimensions, although when higher dimensions are sort, the process can become quite complex, quite quickly. Topic 2 developed on from the theory given within Topic 1. Here a full SVM investigation was given, with comments within the code. As you have no doubt determined, an SVM study is quite involved, yet it is important that you understand the steps taken to conduct such an investigation as they are used frequently within the real world.

Within Topic 3 we mainly focused upon the other Support Vector ideology, in other words SVR. Having completed the topic, it is hoped that you can determine the differences between SVR and SVM and the alternate environments in which they can be applied. The final topic summed up both SVM and SVR: very similar, just quite different. Hopefully, now that we have concluded the lesson this has become clear. If

not, then you are encouraged to further research the topic to gain a much deeper understanding.

Continue to 'Reading list'

Reading List

All reading and resources in this module are provided by the Arden University Library service. The full reading list for this module can be accessed at the link below. More books, journals and databases can be found on the Library Portal. If you need these resources in a different format, our accessibility tools can help.

Full reading list

You can find all the key, essential and wider reading and resources for this module on the dedicated reading list page on the Learning Success Hub.

VIEW READING LIST